

Formal Specification of Linux eBPF Instruction Set Architecture in Sail

Yazhou Tang
tangyazhou@zju.edu.cn
Zhejiang University
Hangzhou, China

Shenghao Yuan*[†]
shenghaoyuan0928@163.com
Zhejiang University
Hangzhou, China

Jean-Pierre Talpin
jean-pierre.talpin@inria.fr
INRIA
Rennes, France

Mingshuai Chen*
m.chen@zju.edu.cn
Zhejiang University
Hangzhou, China

Abstract

eBPF has become a widely used mechanism for extending the Linux kernel, and recent standardization efforts resulted in RFC 9669, the first eBPF ISA standard. However, eBPF still lacks a formal ISA reference that is both executable and presented as an ISA-style document. This paper presents a Sail formalization covering all 153 sequential instructions in the Linux 7.1 eBPF interpreter. For the remaining 28 atomic instructions, it defines encodings, register effects, and memory-effect annotations, but not a concurrent memory semantics. From the same Sail source, we generate a human-readable formal specification document and Rocq definitions. We also provide a handwritten Rocq execution layer consisting of a Rocq driver and a CompCert-backed runtime to execute the generated definitions.

CCS Concepts: • Software and its engineering → Semantics; Operating systems.

Keywords: eBPF, Sail, Mechanized Semantics, Formal Specification, Linux Kernel

ACM Reference Format:

Yazhou Tang, Shenghao Yuan, Jean-Pierre Talpin, and Mingshuai Chen. 2026. Formal Specification of Linux eBPF Instruction Set Architecture in Sail. In *4th Workshop on eBPF and Kernel Extensions (eBPF '26)*, September 29–October 2, 2026, Prague, Czech Republic. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3837779.3838167>

1 Introduction

eBPF (extended Berkeley Packet Filter) [9] was introduced into the Linux kernel in 2014 and has since become a key mechanism for extending kernel functionality. It is now

widely deployed in networking, observability, security monitoring, tracing, and scheduling.

Despite its widespread adoption, eBPF still lacks a crucial component: a precise reference specification with a mathematically rigorous semantics. Such a specification would provide developers and researchers with a reliable foundation for eBPF-related development, analysis, and deployment. For many years, the absence of a standardized eBPF instruction set architecture (ISA) raised concerns among industry stakeholders. For example, vendors interested in eBPF device offloading, such as NVMe developers, expressed concerns about long-term compatibility and specification stability [12]. To address these issues, the eBPF community undertook a standardization effort between 2022 and 2024 [11]. This effort culminated in the publication of RFC 9669 [18], the first official eBPF ISA specification and a major milestone in the evolution of eBPF. However, because RFC 9669 is written in natural language, some semantic details may remain open to interpretation. In contrast, several modern languages and platforms provide formal semantic specifications as part of their official documentation, such as WASM [8].

Previous research has formalized parts of the eBPF semantics using various formal methods and theorem provers. Representative examples include Jitterbugs [14] and JIT-synth [19], using the theorem prover Lean [7]. Other projects use the theorem prover Rocq [5], including the certified RIOT-OS eBPF virtual machine [20], the formal semantics of Solana eBPF [22], CompCert-BPF, which extends the verified C compiler CompCert with an eBPF-like backend [16], and, more recently, the formalization of the Linux eBPF sequential ISA semantics [21]. The latter is the closest to our instruction coverage and provides a mechanized operational semantics in Rocq together with a trace-based validation workflow. Our work is complementary: rather than replacing that semantics, it uses Sail [3], a domain-specific language developed by the REMS project¹, as a single source for an executable model, generated Rocq definitions, and an ISA-style specification document. We therefore do not claim the first formal semantics of Linux eBPF.

Meanwhile, the programming languages and formal methods communities have developed mature approaches for ISA

*Corresponding authors.

[†]The author is with the State Key Laboratory of Blockchain and Data Security & Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou, China



This work is licensed under a Creative Commons Attribution 4.0 International License.

eBPF '26, Prague, Czech Republic

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2911-9/26/09

<https://doi.org/10.1145/3837779.3838167>

¹<https://www.cl.cam.ac.uk/~pes20/remis/>

formalization and documentation. Sail is designed specifically for expressing ISA semantics in a precise and machine-checkable form. It has been successfully applied to major architectures, including x86, ARMv8-A, RISC-V, MIPS, and CHERI-IoT. Notably, the Sail specification of RISC-V has been adopted by the RISC-V Foundation. Despite its success, Sail currently lacks a formal model of the Linux eBPF ISA.

In this work, we develop a formal specification of the Linux eBPF semantics in Sail. Figure 1 summarizes our Sail-centered workflow and its outputs.

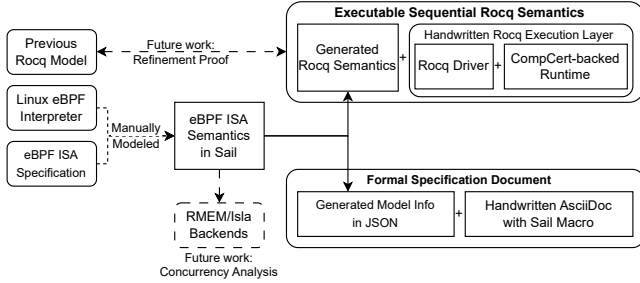


Figure 1. Overview of the Sail-centered workflow, including its generated artifacts, handwritten components, and future refinement and concurrency work.

Figure 1 distinguishes the handwritten and generated parts of our workflow. We manually construct the Sail model from the Linux interpreter and eBPF standardization document. The Sail source generates Rocq semantics, which is combined with a handwritten Rocq execution layer consisting of a Rocq driver and a CompCert-backed runtime. It also generates model information in JSON, which is combined with handwritten AsciiDoc text containing Sail macros to produce the specification document. The dashed paths show future refinement against the previous Rocq model and future concurrency analysis through the RMEM and Isla backends. Generation keeps the derived artifacts synchronized with the Sail source, but does not by itself establish that the Sail model or the handwritten components agree with Linux.

This paper makes the following contributions:

- We present an executable Sail model covering all 153 sequential instructions in the Linux 7.1 eBPF interpreter, together with encodings, register effects, and memory-effect annotations for the remaining 28 atomic instructions.
- From the same Sail source, we generate Rocq definitions and a human-readable, ISA-style PDF specification.

2 Preliminaries

This section provides the necessary background on Sail and CompCert.

2.1 Sail Language

Sail is a domain-specific language developed by the REMS project for describing instruction set architectures. It provides readable ISA specifications while supporting executable models and formal analysis. Its syntax is compact and close to conventional pseudocode, making the resulting specification accessible to users and suitable for review and maintenance. At the same time, Sail provides a mature toolchain for generating definitions for proof assistants such as Rocq, Isabelle/HOL, and Lean, executable C and OCaml code, and semantics-aware documentation. This allows a single specification to serve as a readable reference, an executable model, and a basis for mechanized proof.

2.2 CompCert

The CompCert C verified compiler [13] is a formally verified compiler for a large subset of the C language, targeting architectures such as ARM, PowerPC, RISC-V, and x86. CompCert is mechanized and verified in the Rocq Prover, ensuring that the generated assembly code preserves the behavior prescribed by the semantics of the source C program.

In our backend, we rely on the CompCert memory model to give a precise semantics to eBPF memory operations. This memory model specifies the behavior of operations over memory states, including allocation, deallocation, reads, and writes. A memory state m is organized as a collection of blocks, where each block is an array of abstract bytes. Pointers are represented as pairs (b, i) , where b is a block identifier and i is a byte offset within that block. The model provides several core operations, including `Mem.alloc` for allocation, `Mem.free` for deallocation, `Mem.load` for reading from memory, and `Mem.store` for writing to memory.

3 eBPF Formalization in Sail

This section presents our formal definition of eBPF instructions in Sail, providing the semantic foundation for the precise specification developed later. We choose Linux 7.1 as our authoritative reference, and there are 181 instructions in total in Linux eBPF’s interpreter². Our current formalization covers all 153 sequential instructions in the Linux eBPF instruction set. For the remaining 28 instructions in the atomic instruction family, we provide some basic definitions including encodings, decoded forms, value computation, register writeback, and memory-effect annotations. We leave the integration with a full weak-memory concurrency model to future work.

Our eBPF formalization adopts the standard SOS (Structural Operational Semantics) style [15], which requires the definition of the program state (also called a *configuration*, in subsection 3.1), the top-level execution environment (in subsection 3.2), and the semantics of each eBPF instruction

²<https://elixir.bootlin.com/linux/v7.1/source/kernel/bpf/core.c#L1786>

(in subsection 3.3). Additionally, we give an introduction on the semantics of atomic instructions in subsection 3.4.

3.1 Program State

We begin with the eBPF program *configuration*, which consists of register state, memory state, and auxiliary state.

Register State. The register state consists of a mapping from each eBPF register identifier to its value. The modeled registers include ten general-purpose registers (R0–R9), a local stack pointer R10, an auxiliary register RAX, and the program counter PC. We model R0–R10 and RAX as 64-bit bit-vectors, since these registers store fixed-width machine words. In contrast, PC is represented as a mathematical integer to simplify the semantics design.

```
register r0 : bits(64)
.....
register r10 : bits(64)
register rax : bits(64)
register pc : int
```

Memory State. Our semantics adopts a layered design for memory. At the Sail level, registers store only 64-bit concrete addresses, and memory operations are expressed through abstract memory effects rather than through a fixed concrete memory representation. These effects carry the information needed by the backend to perform the access, including the address, access size, and the access metadata used for ordinary, acquire/release, and atomic read-modify-write accesses.

Concrete execution uses a handwritten Rocq driver together with a CompCert-backed runtime. Within this runtime, the memory implementation maintains a region map from disjoint concrete address ranges to CompCert memory blocks and delegates each legal eBPF memory access to the corresponding CompCert operation, such as Mem.load or Mem.store.

Auxiliary State. The auxiliary state contains three variables used to record the control state during execution.

- For internal calls, we define `call_stack` to store caller's registers.
- For tail calls, we need `tail_call_count` to check whether tail call exceeds the maximum allowed number, and `current_code_id` to fetch correct code.

3.2 Execution Environment for the Sail Model

We now explain the semantics of the eBPF execution environment, which involves all the machinery used for executing eBPF programs. Executing an eBPF program consists of two phases: (1) constructing the initial execution state and (2) executing the program from that state.

3.2.1 State Initialization. The initial execution environment consists of two parts: the initial machine state and several static inputs.

Register Initialization. Before execution starts, PC is set to zero, R1 is initialized to a pointer to the eBPF context, and R10 is initialized to a pointer to the top of the eBPF local stack. All other registers are initialized to zero.

Memory Initialization. The memory accessible to an eBPF program is initialized from a collection of memory regions. Each region is described by its type, concrete base address, size, and initial byte contents. Region types include the eBPF context, the local stack frame, packet data, and eBPF map values. For example, each value in a BPF_MAP_TYPE_HASH map is represented as a separate memory region.

Auxiliary State. The auxiliary execution state is initialized as follows: `call_stack` is an empty list, `tail_call_count` is zero, and `current_code_id` points to the main program.

In addition to the initial machine state, execution also depends on several static inputs.

Program Code. For simplicity, our formalization represents the eBPF program code as abstract lists rather than as memory blocks. The program code comprises two parts: the instruction sequence of the main eBPF program and a jump table for tail calls. The instruction sequence is a list of bytes. The jump table maps integer code identifiers to instruction sequences. A tail-call instruction first queries an eBPF map of type BPF_MAP_TYPE_PROG_ARRAY to obtain a code identifier. It then uses this identifier to fetch the target instruction sequence.

Host Endianness. The host endianness is provided as a static input because it affects both the decoding and execution of eBPF instructions. In the instruction encoding, the `dst_reg` and `src_reg` fields are arranged differently on little-endian and big-endian hosts, and some instructions (*i.e.*, byte-swap) also have endian-dependent semantics.

Tail-call Limit. Following the Linux kernel³, we set the maximum number of tail calls to 33.

3.2.2 Program Execution. After initialization, the program is executed by the Sail function `run_program`, which repeatedly invokes `step_program` under the given step bound. In each step, `step_program` selects the current instruction sequence, fetches the instruction at PC, decodes it, and invokes the corresponding instruction semantics through `execute`, which is described in subsection 3.3.

The execution result of one step is represented by the Sail datatype `ExecutionResult`, which can either indicate normal continuation (Success, for most eBPF instructions), terminate execution (*e.g.*, for the eBPF Exit instruction), or request runtime handling (*e.g.*, for internal and tail calls).

3.3 Instruction Definition in Sail

In Sail, the formalization of an instruction follows a modular three-step structure: defining its abstract syntax, specifying its bidirectional binary encoding and decoding, and implementing its execution semantics. We illustrate this workflow

³<https://elixir.bootlin.com/linux/v7.1/source/include/linux/bpf.h#L2262>

using BPF_SDIV32, which performs 32-bit signed division and stores the result in the destination register.

First, an eBPF instruction is introduced into the global instruction datatype, called `union clause` in Sail.

```
union clause instruction =
  BPF_SDIV32 : (src_kind, regidx, regidx, imm32)
```

For the BPF_SDIV32 example, the `union clause` contains four fields: `src_kind`, which determines whether the source operand is a register (SRC_X) or an immediate (SRC_K); two register indices for the destination and source registers; and a 32-bit immediate value.

Second, the abstract instruction is related to its concrete machine-code representation. Sail’s mapping clause defines such a bidirectional relationship, written using `<->`, between an AST constructor and its binary encoding. The concrete encoding depends on the host endianness [18]; for brevity, we only show the little-endian form here.

For the register-source form of BPF_SDIV32, the mapping is defined as follows:

```
mapping clause encdec = BPF_SDIV32(SRC_X, dst, src,
  0x0000_0000) <-> 0x0000_0000 @ 0x0001
  @ encdec_reg(src) @ encdec_reg(dst)
  @ encdec_alu_code(DIV) @ encdec_src_kind(SRC_X)
  @ encdec_alu_class(ALU32)
```

Here, the `@` operator concatenates bit-vector fields. From the least significant bit to the most significant bit, the little-endian encoding can be read from right to left as follows:

- *Opcode* (bits 0–7): The opcode is composed of the instruction class, source mode, and ALU operation. For the register-source form of BPF_SDIV32, the opcode is `0x3c`: ALU32 contributes `0x04`, SRC_X contributes `0x08`, and DIV contributes `0x30`. The corresponding immediate-source form uses SRC_K and therefore has opcode `0x34`. Auxiliary mappings translate these definitions into bit-vectors.
- *Registers* (bits 8–15): These fields encode the destination and source registers. In the little-endian form shown above, `encdec_reg(dst)` and `encdec_reg(src)` correspond to the `dst_reg` and `src_reg` fields, respectively.
- *Offset* (bits 16–31): The constant `0x0001` occupies the offset field and distinguishes signed division (*i.e.*, `bit[16] = 1`) from unsigned division (*i.e.*, `bit[16] = 0`).
- *Immediate* (bits 32–63): The constant `0x0000_0000` occupies the immediate field, which is unused in this register-source form.

Finally, the operational semantics is defined as a function clause of the `execute` function. When the interpreter encounters a BPF_SDIV32 instruction, it pattern-matches the corresponding AST node and executes the following semantic rule, with some auxiliary functions inlined for readability:

```
function clause execute BPF_SDIV32(srck, dst, src, imm) = {
  // 1. Read the data.
  let dst_v : bits(64) = X(dst);
  let src_v : bits(64) = match srck {
    SRC_X => X(src),
    SRC_K => zero_extend(imm),
  };
  // 2. Operate on the data.
  let out : bits(32) = {
    let d_s = signed(dst_v[31 .. 0]);
    let s_s = signed(src_v[31 .. 0]);
    if s_s == 0
      then 0x0000_0000
    else if (s_s == -1) & (d_s == alu32_signed_min())
      then to_32bits(alu32_signed_min())
    else
      to_32bits(tdiv_int(d_s, s_s))
  };
  // 3. Write the result.
  X(dst) = zero_extend(out);
  PC = PC + 1;
  SUCCESS
}
```

3.4 Atomic Instructions

The eBPF atomic instruction family requires special treatment because its members have concurrency-relevant memory effects. We therefore separate its semantics into two parts. In Sail, we focus on declaring the behavior of these atomic instructions using a set of uninterpreted functions. We leave the remaining aspects (*e.g.*, underlying concurrency memory model and the concurrent execution of these externally uninterpreted functions) for future work.

The current Linux eBPF ISA has two kinds of atomic instructions. The first comprises atomic read-modify-write (RMW) operations, including add, or, and, xor, their fetch variants, xchg, and cmpxchg. The second comprises acquire loads and release stores.

Our Sail model describes the arithmetic and register effects of atomic instructions. Memory effects, however, are emitted through Sail’s concurrency memory interface, following the same architecture used by the Sail RISC-V model [3]. The eBPF model uses 64-bit concrete addresses and attaches two kinds of metadata to each memory effect: (1) AV (Access Variety) identifies the kind of memory action, either a plain load/store (AV_plain) or an atomic read-modify-write access (AV_atomic_rmw); and (2) AS (Access Strength) identifies the ordering strength, either a normal access (AS_normal) or an acquire/release access (AS_rel_or_acq). The following table summarizes how these accesses are classified by the concurrency memory interface.

```
plain load/store ↦ (AV_plain, AS_normal),
atomic RMW access ↦ (AV_atomic_rmw, AS_normal),
load-acquire/store-release ↦ (AV_plain, AS_rel_or_acq).
```

The memory effect’s metadata provides the interface through which future concurrency backends can distinguish plain

accesses, atomic RMW accesses, and acquire/release accesses. This follows the same approach as the reference Sail RISC-V model, where the Sail ISA semantics can be connected to RMEM [17] for operational relaxed-memory exploration and to Isla [4] for axiomatic concurrency analysis. A complete concurrent semantics for eBPF would still need to relate these effects to Linux kernel’s concurrency primitives, for example through an LKMM-style axiomatic treatment [1] or an operational model. In either case, it would need to specify the ordering imposed by acquire/release instructions, and how the read and write effects of a single atomic RMW instruction are grouped into one indivisible update. The current Sail annotations provide the events to be interpreted, but do not by themselves define this full concurrent behavior.

4 Formal Specification for eBPF ISA

Sail offers a major advantage: we can automatically generate human-readable specification documents from a formal ISA model. We leverage this feature to produce a formal eBPF ISA specification directly from our Sail model.

Such a specification typically comprises a collection of entries, one for each eBPF instruction. To keep the specification concise, we group instruction variants that differ only in whether the source operand is an immediate value or a register. Consequently, each instruction entry is a tuple $\langle Id, Desc, Enc, Ope \rangle$:

- *Id* is a string that names the instruction, e.g., `BPF_SDIV32`. It derives directly from the instruction name in the eBPF union clause.
- *Desc* is a set of paragraph strings. These paragraphs record the instruction’s general behavior, and notes about any special corner cases, as well as verifier and JIT-related information.
- *Enc* consists of one or more encoding diagrams. These diagrams graphically represent the encoding format for different source operands. They are extracted from the corresponding Sail mapping clauses.
- *Ope* always includes an operation section generated from the Sail execute clause of the instruction. For most instructions, this operation component fully describes the instruction behavior. For atomic instructions, we additionally provide informal semantics to describe the Linux behavior behind it.

Figure 2 illustrates this generation process for `BPF_SDIV32`. The left-hand side shows the complete Sail definition discussed in subsection 3.3, while the right-hand side shows the corresponding generated specification entry. The documentation comment is rendered as the description, the two mapping clauses produce the immediate-source and register-source encoding diagrams, and the execute clause produces the operation section. The union clause only defines the abstract instruction constructor and therefore has no direct counterpart in the generated document. For readability, the

figure places the documentation comment at the top of the Sail code; in the actual source, it is attached directly to the corresponding execute clause. The generated operation section also expands helper functions called by execute, making the displayed semantics self-contained.

The formal specification is generated in several steps. First, we run ‘`sail --doc`’ to produce a JSON file containing the information extracted from the Sail model. We then use an AsciiDoc file to assemble the generated components into the desired document structure. Finally, we use the AsciiDoctor toolchain together with Sail’s AsciiDoctor plugin [2] to generate the final PDF document.

The encoding diagrams and operation sections are generated from the same Sail definitions used for execution, which reduces the risk that these artifacts diverge. This shared-source relationship does not extend to the manually written descriptive text.

5 Discussion

In this section, we discuss several limitations and directions for future work. We first discuss further validation of our semantics in subsection 5.1. We then use shift semantics as a case study of divergence between the specification and implementations in subsection 5.2. Finally, we discuss how helper-function semantics could be formalized on top of our current model in subsection 5.3 and summarize the model’s maintenance process in subsection 5.4.

5.1 Validation of Semantics

Our current evidence consists of Sail type checking, the model’s executability on concrete eBPF programs, and manual comparison with the interpreter and RFC 9669 during model construction. Sail type checking confirms that the model is well typed, executability makes the semantics inspectable on concrete programs, and manual comparison helps identify transcription errors. Taken together, these activities provide initial confidence in the model, but they do not constitute independent, systematic validation or establish semantic agreement with Linux. A direct next step is to run the BPF Conformance suite [10] on our executable model to check supported instructions. Complementary validation could reuse the trace-based workflow [21] to compare states produced by our model with interpreter traces. Beyond testing, a longer-term direction is to prove a refinement or equivalence theorem relating our Sail sequential semantics—extracted to Rocq—to their Rocq semantics.

5.2 A Shift-Semantics Divergence Case Study

Register-based shifts (LSH, RSH, and ARSH) illustrate a divergence between the specification and implementations. RFC 9669 defines these instructions as using a mask on the shift amount (64-bit shifts use `0x3f`, and 32-bit shifts use `0x1f`), and the Linux eBPF interpreter follows this definition by

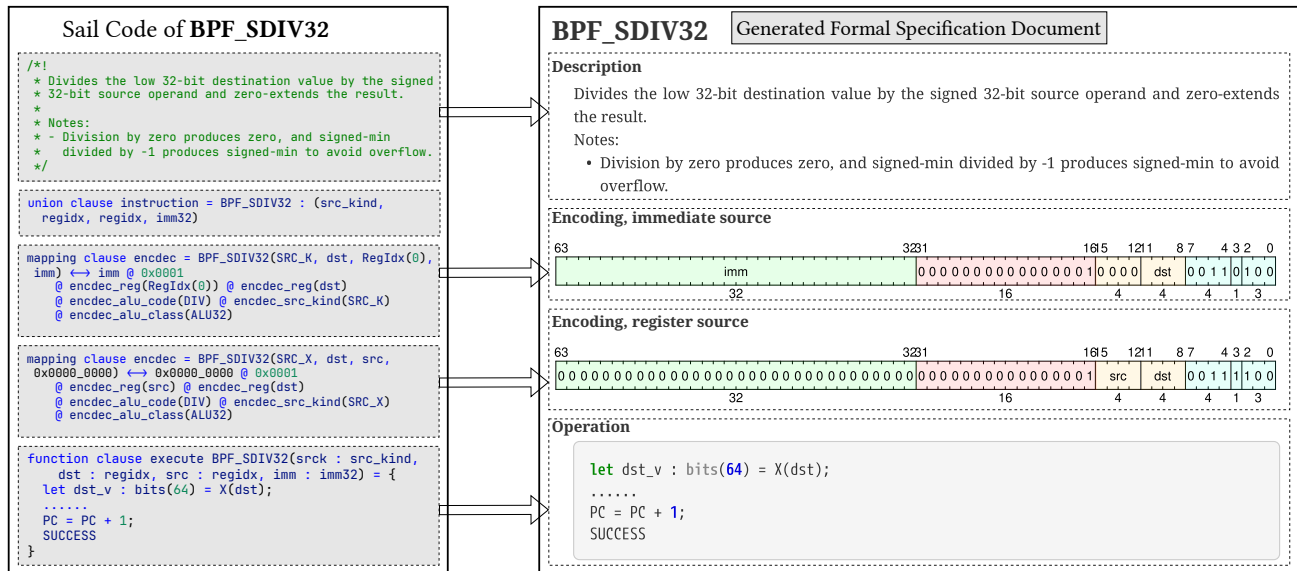


Figure 2. BPF_SDIV32 in the generated specification document. Details of the operation code are omitted.

explicitly masking the source operand before performing the shift.⁴

In contrast, various eBPF JIT backends lower these instructions directly to native shifts without emitting an explicit masking operation. The resulting behavior therefore depends on the target ISA. This agrees with RFC 9669 when the native ISA applies the same mask, but it can produce observable differences on architectures with different out-of-range shift semantics, such as ARM32⁵ and PowerPC⁶. A historical kernel discussion documents that kernel developers deliberately chose not to insert extra masking instructions in JIT-generated code because of performance concerns [6].

Our Sail reference model follows the masked semantics documented in RFC 9669. We therefore treat this behavior as a known specification/implementation divergence rather than a newly discovered kernel defect. Whether JIT backends should enforce the documented semantics, or whether the specification should instead permit architecture-dependent behavior for out-of-range shifts, remains an engineering and standardization question.

5.3 Formalizing Helper Functions

Linux eBPF supports two major classes of kernel functions callable from BPF programs:

1. 211 predefined BPF helper functions⁷, identified by static BPF_FUNC_* IDs and called by a CALL instruction with `src_reg == 0`.
2. BTF-ID-based kernel functions (kfuncs), which are annotated with `__bpf_kfunc`, registered in BTF kfunc ID sets, and called by a CALL instruction with `src_reg == 2`. Unlike predefined helpers, the set of available kfuncs is open-ended and depends on the kernel version and configuration.

In this work, we do not implement an effect handler for eBPF helper calls. The main reason is that helper calls are not merely internal computations in eBPF local stack and registers. Most helpers interact with external objects, such as BPF maps, BPF contexts, packet buffers, sockets, etc.

Our current model provides partial support for such external state on the Rocq side. In particular, we define a generic record for BPF map objects, containing the map identifier, concrete address, map type, key size, value size, maximum number of entries, and map contents, although we currently instantiate it only for array maps, hash maps, and program-array maps. Similarly, the BPF context is not modeled as a separate semantic object, but as an entry in the existing region map. This infrastructure provides a useful starting point for formalizing helpers such as `bpf_map_lookup_elem`, `bpf_map_update_elem`, and `bpf_map_delete_elem`; however, supporting the full helper interface would require additional state components, such as packet state, socket state, as well as oracles for time, randomness, and other environment-dependent values.

⁴<https://elixir.bootlin.com/linux/v7.1/source/kernel/bpf/core.c#L1828>

See <https://developer.arm.com/documentation/dui0473/m/arm-and-thumb-instructions/shift-operations>.

⁶For example, PowerPC word shifts exhibit periodic behavior: depending on the shift count, an out-of-range left shift may either produce zero or behave like a masked shift. See <https://www.ibm.com/docs/en/aix/7.3.0?topic=set-slw-sl-shift-left-word-instruction>.

⁷<https://elixir.bootlin.com/linux/v7.1/source/include/uapi/linux/bpf.h#L5904>

5.4 Maintaining the Model

The present model is versioned against Linux 7.1 rather than automatically synchronized with mainline Linux. Updating it for a new release requires comparing the interpreter and ISA documentation, then revising the affected Sail clauses. The Rocq definitions and specification document must subsequently be regenerated and reviewed. Using a single Sail source avoids separately editing the executable semantics and derived artifacts, but it does not remove the need for manual semantic review. As future work, we plan to explore tooling that translates relevant changes in kernel C code into candidate updates to the Sail model, with the generated changes still subject to semantic review. We have not yet measured the maintenance effort across kernel releases.

6 Conclusion

This paper presents an executable Sail formalization covering all 153 sequential instructions in the Linux 7.1 eBPF interpreter. For the remaining 28 atomic instructions, the model defines encodings, register effects, and memory-effect annotations, but not a concurrent memory semantics. From the same Sail source, we generate Rocq definitions and a formal specification document. This shows that Sail can serve as a practical foundation for both executable eBPF semantics and an ISA-style specification. The Sail model and the generated specification document are available at:

<https://github.com/OpenSourceVerif/sail-ebpf/tree/ebpf26-artifact>

Acknowledgments

This work has been partially funded by the ZJNSF Major Program (No. LD24F020013), by the Zhejiang Pioneer (Jianbing) Project (2024C01032), by the NSFC (No. 62502431, No. 62532012, and No. 62572427), by the Zhejiang Key Laboratory Project (No. 2024E10001), and by the Fundamental Research Funds for the Central Universities of China (No. 226-2024-00140). We would like to thank the anonymous reviewers for their valuable feedback.

References

- [1] Jade Alglave, Luc Maranget, Paul E McKenney, Andrea Parri, and Alan Stern. 2018. Frightening small children and disconcerting grown-ups: Concurrency in the linux kernel. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, New York, NY, USA, 405–418. doi:10.1145/3173162.3177156
- [2] Alasdair Armstrong. 2025. Sail Asciidoctor Plugin. <https://github.com/Alasdair/asciidoctor-sail> Accessed: 2026-06-19.
- [3] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E Gray, Robert M Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, et al. 2019. ISA semantics for ARMv8-a, RISC-v, and CHERI-MIPS. *Proc. ACM Program. Lang.* 3, POPL (2019), 1–31. doi:10.1145/3290384
- [4] Alasdair Armstrong, Brian Campbell, Ben Simner, Christopher Pulte, and Peter Sewell. 2021. Isla: Integrating Full-Scale ISA Semantics and Axiomatic Concurrency Models. In *Computer Aided Verification*, Alexandra Silva and K. Rustan M. Leino (Eds.). Springer International Publishing, Cham, 303–316. doi:10.1007/978-3-030-81685-8_14
- [5] Yves Bertot and Pierre Castéran. 2013. *Interactive theorem proving and program development: Coq'Art: the calculus of inductive constructions*. Springer Science & Business Media. doi:10.1007/978-3-662-07964-5
- [6] Daniel Borkmann. 2021. bpf: Fix up register-based shifts in interpreter to silence KUBSAN. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=28131e9d933339a92f78e7ab6429f4aa007061c> Accessed: 2026-06-19.
- [7] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. The Lean Theorem Prover (System Description). In *Automated Deduction - CADE-25*. Springer International Publishing, Cham, 378–388. doi:10.1007/978-3-319-21401-6_26
- [8] Andreas Rossberg (editor). 2026. WebAssembly Specification (Release 3.0). <https://webassembly.github.io/spec/core/> Accessed: 2026-06-19.
- [9] Matt Fleming. 2017. A Thorough Introduction to eBPF. <https://lwn.net/Articles/740157/> Accessed: 2026-06-19.
- [10] Alan Jowett and contributors. 2026. BPF Conformance. https://github.com/Alan-Jowett/bpf_conformance Accessed: 2026-08-13.
- [11] The kernel development community. 2025. BPF Instruction Set Architecture (ISA). <https://docs.kernel.org/bpf/standardization/instruction-set.html> Accessed: 2026-06-19.
- [12] Suresh Krishnan and David Vernet. 2024. Core BPF ecosystem standardized in RFC 9669. <https://www.ietf.org/blog/bpf-rfc9669/> Accessed: 2026-06-19.
- [13] Xavier Leroy. 2009. Formal verification of a realistic compiler. *Commun. ACM* 52, 7 (July 2009), 107–115. doi:10.1145/1538788.1538814
- [14] Luke Nelson, Jacob Van Geffen, Emina Torlak, and Xi Wang. 2020. Specification and verification in the field: Applying formal methods to BPF just-in-time compilers in the Linux kernel. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, USA, 41–61.
- [15] Gordon D. Plotkin. 1981. *A Structural Approach to Operational Semantics*. Technical Report DAIMI FN-19. Computer Science Department, Aarhus University, Aarhus, Denmark.
- [16] Swarn Priya, Frédéric Besson, Connor Sughrie, Tim Steenvoorden, Jamie Fulford, Freek Verbeek, and Binoy Ravindran. 2025. BeePL: Correct-by-compilation kernel extensions. arXiv:2507.09883 [cs.PL]
- [17] Christopher Pulte, Shaked Flur, Will Deacon, Jon French, Susmit Sarkar, and Peter Sewell. 2017. Simplifying ARM concurrency: multicopy-atomic axiomatic and operational models for ARMv8. *Proc. ACM Program. Lang.* 2, POPL (2017), 1–29. doi:10.1145/3158107
- [18] Dave Thaler. 2024. BPF Instruction Set Architecture (ISA). RFC 9669. doi:10.17487/RFC9669
- [19] Jacob Van Geffen, Luke Nelson, Isil Dillig, Xi Wang, and Emina Torlak. 2020. Synthesizing JIT Compilers for In-Kernel DSLs. In *Computer Aided Verification*. Springer International Publishing, Cham, 564–586. doi:10.1007/978-3-030-53291-8_29
- [20] Shenghao Yuan, Frédéric Besson, Jean-Pierre Talpin, Samuel Hym, Koen Zandberg, and Emmanuel Baccelli. 2022. End-to-End Mechanized Proof of an eBPF Virtual Machine for Micro-controllers. In *Computer Aided Verification*. Springer International Publishing, Cham, 293–316. doi:10.1007/978-3-031-13188-2_15
- [21] Shenghao Yuan, Yazhou Tang, Tianci Cao, Frédéric Besson, Jean-Pierre Talpin, and Mingshuai Chen. 2026. Formalizing the Linux eBPF Core ISA: A Mechanized Operational Semantics and Its Real-World Applications. *Proc. ACM Program. Lang.* 10, OOPSLA2 (2026). <https://shenghaoyuan.github.io/publications/OOPSLA2026.pdf> To appear.
- [22] Shenghao Yuan, Zhuoruo Zhang, Jiayi Lu, David Sanan, Rui Chang, and Yongwang Zhao. 2025. A Complete Formal Semantics of eBPF Instruction Set Architecture for Solana. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 80 (April 2025), 27 pages. doi:10.1145/3720414